
Parte 4: El Corazón de la Lógica (CarritoNotifier)

1. El Concepto: Sincronización Doble

El Notifier debe realizar dos tareas simultáneas cada vez que el usuario toca un botón:

1. **En Memoria:** Actualizar el estado de la UI inmediatamente para que la app se sienta rápida.
2. **En Disco:** Guardar el cambio en Isar para que sea permanente.

2. Implementación: Notifier del Carrito (`providers/carrito_provider.dart`)

Este provider es un poco más complejo porque maneja operaciones de escritura.

```
import 'package:riverpod_annotation/riverpod_annotation.dart';
import 'package:isar/isar.dart';
import '../models/producto.dart';
import '../models/carrito_item.dart';
import '../services/db_service.dart';

part 'carrito_provider.g.dart';

@riverpod
class Carrito extends _$Carrito {

  @override
  Future<List<CarritoItem>> build() async {
    // Al iniciar, leemos lo que ya estaba guardado en el carrito
    return _fetchItems();
  }

  Future<List<CarritoItem>> _fetchItems() async {
    // Importante: .load() carga los datos del producto vinculado (Link)
    final items = await DBService.isar.carritoItems.where().findAll();
    for (var item in items) {
      await item.producto.load();
    }
    return items;
  }

  Future<void> agregarProducto(Producto producto) async {
    final isar = DBService.isar;

    await isar.writeTxn(() async {
      // 1. Verificamos si el producto ya está en el carrito
      final existente = await isar.carritoItems
        .filter()
        .producto((q) => q.idEqualTo(producto.id))
        .findFirst();
```

```

    if (existente != null) {
      // 2. Si existe, aumentamos cantidad
      existente.cantidad++;
      await isar.carritoItems.put(existente);
    } else {
      // 3. Si es nuevo, creamos el item y vinculamos el producto
      final nuevoItem = CarritoItem()..cantidad = 1;
      nuevoItem.producto.value = producto;
      await isar.carritoItems.put(nuevoItem);
      await nuevoItem.producto.save(); // Guarda la relación
    }
  });

  // 4. Refrescamos el estado local
  ref.invalidateSelf();
}

Future<void> eliminarItem(int id) async {
  await DBService.isar.writeTxn(() async {
    await DBService.isar.carritoItems.delete(id);
  });
  ref.invalidateSelf();
}
}

```

3. Teoría detrás del código: Los IsarLinks

Es vital entender el manejo de los **Links**, ya que es un concepto puramente de ingeniería de datos:

- **nuevoItem.producto.value = producto**: Aquí no estamos copiando el nombre y precio del producto. Estamos diciendo: "Este item del carrito apunta al ID del producto X".
- **await nuevoItem.producto.save()**: En Isar, después de guardar el objeto principal, debemos guardar explícitamente el vínculo (la relación).
- **await item.producto.load()**: Por defecto, Isar no carga los objetos vinculados para ahorrar memoria (Lazy Loading). Debemos cargarlos manualmente cuando necesitemos mostrar el nombre o precio del producto en la UI.

4. Lógica de Negocio Avanzada: El "Total"

Para que la UI no tenga que calcular el total (lo cual es una mala práctica), podemos crear un **Provider Derivado**. Este provider se actualizará automáticamente cada vez que el carrito cambie.

```

@riverpod
double carritoTotal(Ref ref) {
  final carritoAsync = ref.watch(carritoProvider);

  return carritoAsync.maybeWhen(
    data: (items) => items.fold(0, (sum, item) => sum +

```

```
(item.producto.value!.precio * item.cantidad)),  
  orElse: () => 0.0,  
);  
}
```

5. Análisis

Considera este escenario:

"Si un alumno olvida usar `ref.invalidateSelf()`, ¿qué pasaría?"

Respuesta: El dato se guardaría correctamente en la base de datos (persistencia), pero el usuario no vería el cambio en la pantalla hasta que reinicie la app. Esto demuestra la separación entre el **Estado de Datos** (Isar) y el **Estado de UI** (Riverpod).
